ZHENGZHOU UNIVERSITY

School of Computer and Artificial Intelligence

Design and Analysis of Algorithms

Lab Practice 04 | Big-O Equations, Factorial & Fibonacci

Instructor: Dr. Hussain **Duration:** 90 minutes **Language:** C++ or Python **Type:** Hands-on, no marks

1. Learning objectives and lab rules

Identify the dominant term in time-complexity equations; distinguish iteration from recursion; implement factorial and Fibonacci using both approaches; trace calls and compare time and auxiliary space.

Lab rule: Write pseudocode and program code manually. Do not copy and paste solutions. Explain your own results. Use small test inputs, especially for recursive Fibonacci.

2. Time plan

Minutes	Activity
0-15	Task 1: Big-O equations
15-25	Task 2: Base case and recursive calls
25-40	Task 3: Factorial, two approaches
40-60	Task 4: Fibonacci, two approaches
60-75	Task 5: Manual tracing and complexity
75-85	Task 6: Test and compare
85-90	Submission

3. Task 1 - Identify Big-O from equations (15 min)

Step A. For each expression, underline the fastest-growing term as n becomes large. **Step B.** Ignore constant multipliers and lower-order terms. **Step C.** Fill in Big-O and write a one-sentence explanation for each.

Expression	Dominant term	Big-O
$T(n) = 5n + 10$	_____	_____
$T(n) = 3n^2 + 2n + 5$	_____	_____
$T(n) = 7$	_____	_____
$T(n) = 4 \log\blacksquare(n) + 3$	_____	_____
$T(n) = 2\blacksquare + n^2$	_____	_____

Check: Explain why 5n + 10 and 100n + 2 have the same asymptotic growth class.

4. Task 2 - Understand recursion (10 min)

Step A. Define iteration (repetition with a loop) and recursion (a function calling itself). **Step B.** Explain the base case: the condition that stops recursive calls. **Step C.** For factorial, identify the base case in F(0) = 1 and the recursive rule F(n) = n × F(n-1) for n > 0.

Expected output: Two short definitions and one labeled example of a base case and recursive call.

5. Task 3 - Factorial using iteration and recursion (15 min)

Step A. Write pseudocode for iterative factorial using a loop. **Step B.** Write pseudocode for recursive factorial using the base case n = 0 (or n = 1). **Step C.** Implement both functions in C++ or Python. **Step D.** Run n = 0, 1, 5 and 7; print the outputs of both functions side by side.

Example: 5! = 5 × 4 × 3 × 2 × 1 = 120. Reject negative inputs or handle them explicitly. In C++, choose an integer type that safely holds the tested results.

n	Iterative result	Recursive result
0	_____	_____
1	_____	_____
5	_____	_____
7	_____	_____

6. Task 4 - Fibonacci using iteration and recursion (20 min)

Definition: F(0) = 0, F(1) = 1, and F(n) = F(n-1) + F(n-2) for n ≥ 2.

Step A. Write iterative pseudocode using two variables to store the previous values. **Step B.** Write naive recursive pseudocode with two base cases (n = 0 and n = 1). **Step C.** Implement both as functions returning F(n), not just printing a sequence. **Step D.** To display the first n terms, call each function for indices 0 through n-1. Test n = 5, 8 and 10.

Expected first 8 terms: 0, 1, 1, 2, 3, 5, 8, 13. The eighth term count is based on indices 0 to 7.

Important: Keep n small for naive recursive Fibonacci because it repeats many calculations.

7. Task 5 - Manual trace and complexity (15 min)

Step A. Draw the recursive call chain for factorial(4). **Step B.** Draw a call tree for fibonacci(5); show repeated calls such as fibonacci(3). **Step C.** Count total function invocations in each trace (including base cases). **Step D.** Complete the comparison table below for the specified implementations.

Implementation	Time complexity	Auxiliary space
Factorial, iterative	_____	_____
Factorial, recursive	_____	_____
Fibonacci, iterative (F(n))	_____	_____
Fibonacci, naive recursive	_____	_____

Hint: The iterative Fibonacci function keeps only two previous values; the naive recursive version creates repeated branches. For complexity analysis, assume constant-time fixed-size arithmetic and ignore integer overflow.

8. Task 6 - Test and compare (10 min)

Step A. Run both factorial functions for the required inputs and confirm equal results. **Step B.** Run both Fibonacci functions for n = 5, 8, 10 and confirm the same sequence. **Step C.** Record at least one screenshot of each algorithm pair. **Step D.** Answer: (1) Why does recursion require a base case? (2) Which Fibonacci method repeats calculations? (3) Which Fibonacci method would you choose for large n, and why?

9. Final task and submission (5 min)

Create a folder named **StudentID_Name_DAA_Lab04**. Include: (1) factorial program, (2) Fibonacci program, and (3) **Lab04_Report.pdf** containing your Big-O table, pseudocode for all four implementations, manual traces, complexity comparison, test outputs and short answers. Submit to the class monitor before the end of class.

Reminder: This is practice only. No marks or grading. All submitted work must be your own.